

Backend Architecture & Engineering Direction — AI Grant Assistant for NGOs

Product Backend Vision

The backend should be designed as a clean, modular, AI-ready service layer capable of supporting:

- intelligent grant matching
- proposal generation
- Kanban workflow management
- NGO profile processing
- semantic search
- scalable AI integrations

The architecture should prioritize:

- rapid development
- maintainability
- extensibility
- clean API design
- AI orchestration
- hackathon execution speed

The backend should feel:

- modern
 - structured
 - production-oriented
 - AI-native
-

Core Backend Stack

Main Framework

- Django
 - Django Ninja
 - Python
-

Why Django + Ninja

Django provides:

- ORM

- authentication
 - admin panel
 - security
 - database management
 - scalability foundation
-

Django Ninja provides:

- fast modern APIs
- type-safe schemas
- automatic OpenAPI docs
- clean request/response validation
- FastAPI-like developer experience

This combination gives:

- enterprise-level structure
 - startup-level development speed
-

Backend Architecture Philosophy

The backend should follow:

- modular monolith architecture
- service-oriented internal structure
- clean separation of concerns

Avoid:

- microservices
 - unnecessary infrastructure complexity
 - overengineering
-

Recommended Backend Structure

```
backend/
|
├─ manage.py
├─ requirements/
├─ config/
|
├─ apps/
|   └─ authentication/
|   └─ organizations/
|   └─ grants/
```

```
|   ├── proposals/
|   ├── ai/
|   ├── matching/
|   └── analytics/
|
|   ├── core/
|   │   ├── settings/
|   │   ├── database/
|   │   ├── security/
|   │   ├── middleware/
|   │   └── utilities/
|   │
|   ├── services/
|   ├── prompts/
|   ├── vector_store/
|   └── media/
```

API Layer

Framework

- Django Ninja

API Style

- REST API
- typed schemas
- modular routers

Example API Structure

```
/api/v1/
  /organizations
  /grants
  /proposals
  /matching
  /ai
```

Authentication System

Recommended

- JWT Authentication
 - Django auth system
 - django-ninja-jwt
-

Features

- login
 - registration
 - organization accounts
 - protected routes
-

Database Layer

Recommended Database

- PostgreSQL
-

ORM

- Django ORM
-

Main Models

Organization

Stores:

- mission
 - goals
 - nonprofit categories
 - historical projects
 - funding needs
-

Grant

Stores:

- title
 - funding amount
 - requirements
 - deadline
 - tags
 - source
-

Proposal

Stores:

- AI-generated draft
 - status
 - edits
 - linked grant
 - linked organization
-

User

Stores:

- account data
 - permissions
 - organization relationship
-

AI Layer Architecture

This is the most important conceptual backend layer.

AI Service Layer

The AI system should be isolated into:

- prompt services
 - generation services
 - evaluation services
 - matching services
-

Recommended Structure

```
apps/ai/  
  services/  
  prompts/  
  providers/  
  pipelines/
```

AI Providers

Recommended

- watsonx.ai

Optional:

- OpenAI
 - Anthropic
-

AI Workflows

Grant Matching Agent

Input

- NGO mission
 - historical projects
 - categories
-

Output

- compatibility score
 - relevant grants
-

Proposal Generation Agent

Input

- grant requirements
 - organization profile
-

Output

- structured proposal draft
-

Proposal Improvement Agent

Capabilities:

- rewrite
 - summarize
 - improve clarity
 - increase persuasiveness
-

Prompt Engineering Structure

Prompts should be versioned and modular.

Recommended Prompt Structure

```
prompts/  
  matching/  
  proposal_generation/  
  proposal_review/
```

Semantic Search Layer

Recommended

- ChromaDB OR
 - FAISS
-

Purpose

- semantic grant matching
 - similarity search
 - contextual recommendations
-

Embedding Workflow

```
graph TD; A[Grant Data] --> B[Embedding Model]; B --> C[Vector Database]; C --> D[Similarity Search];
```

Background Tasks

Recommended

- Celery
- Redis

Used for

- AI generation tasks
- embeddings generation
- grant indexing
- async processing

File Handling

Features

- PDF uploads
- document parsing
- proposal exports

Recommended Libraries

- PyPDF2
 - pdfplumber
 - python-docx
-

External Data Sources

Grant APIs

Potential integrations:

- Grants.gov
 - EU Open Data
 - local nonprofit funding APIs
-

MVP Recommendation

Use:

- mocked grant datasets
- local JSON/CSV data

Focus should remain on:

- workflow
 - UX
 - AI interaction
-

Admin Panel

One major advantage of Django.

Use Django Admin for:

- managing grants
- organization moderation
- proposal inspection
- analytics

This dramatically speeds development.

API Documentation

Automatically generated with:

- Django Ninja OpenAPI

Important for:

- hackathon demos
 - maintainability
 - frontend integration
-

Security Considerations

Include:

- environment variables
- rate limiting
- secure API keys
- protected AI endpoints

Avoid:

- exposing prompts
 - exposing provider secrets
-

Deployment Stack

Backend Hosting

Recommended:

- Railway
 - Render
 - Fly.io
-

Database Hosting

- Neon PostgreSQL OR
 - Supabase PostgreSQL
-

Storage

Optional:

- Cloudinary
- Supabase Storage

Development Priorities for Hackathon

Focus on:

- AI workflows
- clean API architecture
- fast iteration
- proposal generation quality
- semantic matching

Avoid spending excessive time on:

- enterprise auth
 - advanced permissions
 - infrastructure scaling
-

Recommended Development Workflow

Step 1

Create Django project.

Step 2

Setup Ninja API routes.

Step 3

Build organization/grant models.

Step 4

Integrate AI services.

Step 5

Implement semantic matching.

Step 6

Build proposal generation pipeline.

Step 7

Connect frontend.

Final Backend Goal

The backend should feel like: "A modern AI orchestration platform for nonprofit funding workflows."

Not: "A collection of disconnected AI endpoints."